

Table des matières

<i>Algorithme de Dijkstra (coûts ≥ 0)</i>	2
Problème.....	2
Principe.....	2
Algorithme de Dijkstra.....	4
Initialisation.....	4
Itération	5
Arrêt.....	5
Exemple.....	8
Initialisation.....	8
Iterations.....	8
Vérification :	10
Complexité de l'algorithme.....	11
<i>Algorithme de Bellman (coûts quelconques)</i>	12
Hypothèse.....	12
Principe.....	12
Test d'arrêt	12
Exemple.....	15
Initialisation.....	15
Itérations.....	15
<i>Algorithme de Floyd (coûts quelconques)</i>	17
But	18
Principe.....	18
Etapas de l'algorithme.....	19
Initialisation.....	19
Iterations.....	19
Test d'arrêt	19
Algorithme de Floyd	20
Matrice des Prédecesseurs.....	22
Exemple.....	23
Exemple de chemin	24
Détection de circuit absorbant.....	24
But	25
Théorème.....	25
Preuve.....	26

ALGORITHME DE DIJKSTRA (COÛTS ≥ 0)*Problème*

Calculer un plus court chemin entre un sommet de départ s et tous les autres sommets accessibles depuis s dans un graphe valué dont les valeurs des arcs **sont positives ou nulles**.

On obtient alors une arborescence de racine s formée par ces plus courts chemins.

Principe

Soit $G=(S, A, V)$ un graphe valué avec $n = \text{card}(S)$ nombre de sommets.

Soit s un sommet de départ de G .

On construit progressivement un ensemble CC de sommets t pour lesquels on connaît un plus court chemin de s vers t .

Au départ

$CC \leftarrow \{s\}$

A chaque itération

$CC \leftarrow CC + \{t\}$ tel que $ppdistance(s, t)$ et $pere(s, t)$ sont connus

Arrêt

On s'arrête lorsque CC contient tous les sommets accessibles depuis s .

Soit $M = S \setminus CC$

$\forall t \in M$: on appelle chemin de s à t dans CC tout chemin de s à t ne comportant que des éléments dans CC sauf t .

On note :

$distance_{s, cc(t)}$ = le coût d'un plus court chemin de s à t dans CC

$pred_{s, cc(t)}$ = le prédécesseur de t dans ce plus court chemin

A chaque étape :

On choisit un sommet m de M tel que :

$$Distance_{s, cc(m)} = \inf_{t \in M} \{distance_{s, cc(t)}\}$$

On supprime m de M et on l'ajoute à CC .

Algorithme de Dijkstra

On utilise un ensemble M de sommets de G et deux tableaux D et P .
 s = sommet de départ

M = ensemble qui contient les sommets qui ne sont pas encore accessibles depuis s

D = tableau de coût indicés par les sommets du graphe.

P = tableau des prédécesseurs de sommets indicés par les sommets du graphe.

$D[t]$ = le coût d'un plus court chemin de s à t dans CC

$P[t]$ = sommet prédécesseur de t du plus court chemin

Initialisation

$$M \leftarrow S \setminus \{s\}$$

Pour tout $t \in M$, on initialise

$$\left. \begin{array}{l} D[t] \leftarrow coût(s, t) \\ P[t] \leftarrow s \end{array} \right\} \text{ssi } \exists (s, t) \in A$$

$$\left. \begin{array}{l} D[t] \leftarrow \infty \\ P[t] \leftarrow -1 \end{array} \right\} \text{ssi } (s, t) \notin A$$

On suppose que :

$D[s] = 0$ c.à.d $coût(s, s) = 0$ (on suppose les graphes sans boucles)

$$P[s] = s$$

Itération

A chaque étape on calcule

$$m \in M / D[m] = \min_{t \in M} (D[t])$$

$$M \leftarrow M \setminus \{m\}$$

Pour tout $t \in \Gamma(m)$ **tel que** $t \in M$ **faire**

$$\text{tmp} \leftarrow D[m] + \text{coût}(m, t)$$

Si $\text{tmp} < D[t]$ **alors**

$$D[t] \leftarrow \text{tmp}$$

$$P[t] \leftarrow m$$

FinSi

FinPour

Arrêt

L'algorithme s'arrête pour l'un des cas suivants :

- a) $M = \text{ensemble vide} \rightarrow$ ce cas correspond au cas où tous les sommets de S sont accessibles.
- b) $D[m] = \infty \rightarrow$ les sommets qui restent dans M sont inaccessibles depuis s .

Rappel : On suppose que les sommets sont numérotés de 1 à n

Procédure Dijkstra(s, G, D, P)

$G=(S, A, V)$ graphe sans circuit valué par des coûts quelconques.

s = sommet de départ

Retourne (D et P) un plus court chemin de s à chacun des autres sommets de G .

Début

/* Initialisation */

$$M \leftarrow S \setminus \{s\}$$

Pour tout sommet $t \in M$ **faire**

Si $(s, t) \in A$ **alors**

$$D[t] \leftarrow \text{Coût}(s, t)$$

$$P[t] \leftarrow s$$

Sinon

$$D[t] \leftarrow \infty$$

$$P[t] \leftarrow -1$$

Finsi

Finpour

$$D[s] \leftarrow 0$$

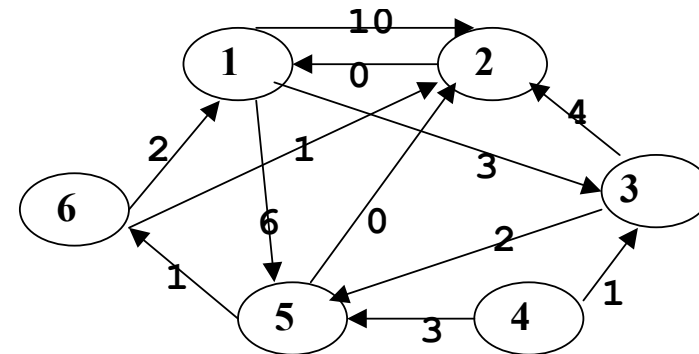
$$P[s] \leftarrow s$$

/* Itérations */estAccessible $\leftarrow$ vrai**Tantque** estAccessible et non estVide(M) **faire**// Recherche un sommet $m \in M$ tel que $D[m]$ est minimal $m \leftarrow \text{recherche}(M, D)$ si $D[m] = \infty$ alors

// les sommets de M sont inaccessibles depuis s

estAccessible $\leftarrow$ faux

sinon

 $M \leftarrow M \setminus \{m\}$ **Pour** tout $t \in \Gamma(m)$ tel que $t \in M$ **faire** $\text{tmp} \leftarrow D[m] + \text{coût}(m, t)$ **Si** $\text{tmp} < D[t]$ alors $D[t] \leftarrow \text{tmp}$ $P[t] \leftarrow m$ **Finsi****Finpour****Finsi****Fintq****Fin****Exemple**Sommet départ = 1

Sommet s	1	2	3	4	5	6
$\Gamma(s)$	{2, 3, 5}	{1}	{2, 5}	{3, 5}	{2, 6}	{1, 2}

Initialisation $M = S \setminus \{1\} = \{2, 3, 4, 5, 6\}$ $\Gamma(1) = \{2, 3, 5\}$

D=	0	10	3	∞	6	∞
P=	1	1	1	-1	1	-1
	1	2	3	4	5	6

Iterations**Iter 1** $M = M \setminus \{3\} = \{2, 4, 5, 6\}$ $\Gamma(3) = \{2, 5\}$ En effet, $3 \in M$ et $D[3]=3$ est minimale

D=	0	7	3	∞	5	∞
P=	1	3	1	-1	3	-1
	1	2	3	4	5	6

2 $\in M$ $D[3] + \text{cout}(3, 2) = 3 + 4 = 7 < D[2] = 10$ on effectue le changement5 $\in M$ $D[3] + \text{cout}(3, 5) = 3 + 2 = 5 < D[5] = 6$ on effectue le changement

Iter 2 $M = M \setminus \{5\} = \{2, 4, 6\}$ $\Gamma(5) = \{2, 6\}$

En effet, $5 \in M$ et $D[5] = 5$ est minimale

$2 \in M$ $D[5] + \text{cout}(5, 2) = 5 + 0 = 5 < D[2] = 7$ on effectue le changement

$6 \in M$ $D[5] + \text{cout}(5, 6) = 5 + 1 = 6 < D[6] = \infty$ on effectue le changement

D=	0	5	3	∞	5	6
P=	1	5	1	-1	3	5
	1	2	3	4	5	6

Iter 3 $M = M \setminus \{2\} = \{4, 6\}$

En effet, $2 \in M$ et $D[2] = 5$ est minimale

Pas changement puisque $\Gamma(2) = \{1\}$ n'appartiennent pas à M

D=	0	5	3	∞	5	6
P=	1	5	1	-1	3	5
	1	2	3	4	5	6

Iter 4 $M = M \setminus \{6\} = \{4\}$

En effet, $6 \in M$ et $D[6] = 6$ est minimale

Pas changement puisque $\Gamma(6) = \{1, 2\}$ n'appartiennent pas à M

D=	0	5	3	∞	5	6
P=	1	5	1	-1	3	5
	1	2	3	4	5	6

L'algorithme s'arrête car $\text{choisir-min}(M, D) = 4$ et $D[4] = \infty$

Autrement-dit, le sommet 4 n'est pas accessible depuis le sommet de départ s=1

Vérification :

D=	0	5	3	∞	5	6
P=	1	5	1	-1	3	5
	1	2	3	4	5	6

$PCC(1, 1) = D[1] = 0$

$\text{chemin}(1, 1) = \{\}$

$PCC(1, 2) = D[2] = 5$

$\text{chemin}(1, 2) = 1 \rightarrow 3 \rightarrow 5 \rightarrow 2$



$PCC(1, 3) = D[3] = 3$

$\text{chemin}(1, 3) = 1 \rightarrow 3$

$PCC(1, 4) = D[4] = \text{l'infini}$

le 4 n'est pas accessible à partir de 1

$PCC(1, 5) = D[5] = 5$

$\text{chemin}(1, 5) = 1 \rightarrow 3 \rightarrow 5$

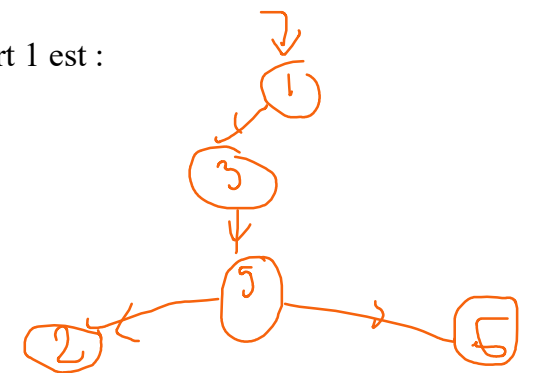
$PCC(1, 6) = D[6] = 6$

$\text{chemin}(1, 6) = 1 \rightarrow 3 \rightarrow 5 \rightarrow 6$



L'arborescence du PCC à

Partir du sommet de départ 1 est :



Complexité de l'algorithme

Il est de l'ordre de n^2 en effet :

- n itérations de la boucle tant que
- et pour chaque itération, on effectue la recherche du minimum qu'est d'ordre de n

On peut réduire la complexité en utilisant des structures de données plus performants :

- Liste triée.
- Arbre binaire de recherche.
- **File de priorité.**

ALGORITHME DE BELLMAN (COÛTS QUELCONQUES)

Hypothèse

Cet algorithme est valable pour les graphes sans circuit, valués par des coûts quelconques.

Il permet de trouver un plus court chemin d'un sommet s (sommet de départ) à tous les autres sommets ainsi que son coût.

Cet algorithme traite les graphes sans circuits.

Principe

Un plus court chemin de s à t doit passer par l'un des prédécesseurs m de t pour lequel la ppdistance (s, m) + coût(m, t) est minimale.

Soit M = l'ensemble des sommets t pour lesquels ppdistance(s, t) n'est pas encore connue.

- ♦ On ne calcule ppdistance(s, t) que lorsque tous les prédécesseurs de t sont dans $CC = S \setminus M$ = sommets explorés.
- ♦ Il est donc nécessaire de connaître, pour tout sommet t, son demi degré intérieur et ses prédécesseurs.
- ♦ L'algorithme choisit dans M un sommet t tel que ses prédécesseurs sont dans CC :
$$D[t] \leftarrow \text{Minimum } \{ D[m] + \text{coût}(m, t) \text{ tel que } m \in \Gamma^{-1}(t) \} \text{ (I)}$$
$$P[t] \leftarrow m \text{ tel que m vérifié (I)}$$

Test d'arrêt

L'algorithme se termine lorsque tous les sommets sont explorés.

Procédure Bellman (s, G, D, P)

$G=(S, A, V)$ graphe sans circuit valué par des coûts quelconques.

s = sommet de départ

L'algorithme retourne (D, P) un plus court chemin de s à chacun des autres sommets de G.

Début

```
// initialisation
```

$$M \leftarrow S \setminus \{s\}$$

Pour tout sommet t de M faire

$$D[t] \leftarrow \infty$$

$$P[t] \leftarrow -1$$

FinPour

$$D[s] \leftarrow 0$$

$$P[s] \leftarrow s$$

// itérations

Tantque non vide (M) faire

```
// t est un sommet non marqué dont tous les
prédécesseurs sont marqués.
```

$$t \leftarrow \text{choisir-suivant}(M, G)$$

$$M \leftarrow M \setminus \{t\}$$

```
// on cherche sommet m = prédécesseur de t
tel que  $D[m] + \text{coût}(m, t)$  soit minimale
```

$$m \leftarrow \text{Premier Prédécesseur de } t$$

$$\min \leftarrow D[m] + \text{coût}(m, t)$$

Pour u variant du deuxième au dernier prédécesseur de t faire

$$\text{tmp} \leftarrow D[u] + \text{coût}(u, t)$$

si tmp < min alors

$$\min \leftarrow \text{tmp}$$

$$m \leftarrow u$$

finsi

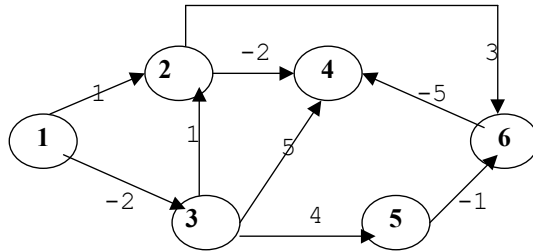
Finpour

$$D[t] \leftarrow \min$$

$$P[t] \leftarrow m$$

Fintantque

FIN

Exemple

Sommet de départ = 1 tel que $\Gamma^{-1}(1) = \{\}$

CC={1}

Sommet s	1	2	3	4	5	6
$\Gamma^{-1}(s)$	{}	{1, 3}	{1}	{2, 3, 6}	{3}	{2, 5}

Initialisation

$M = S \setminus \{1\} = \{2, 3, 4, 5, 6\}$

D=	0	∞	∞	∞	∞	∞
P=	1	-1	-1	-1	-1	-1
	1	2	3	4	5	6

Itérations

Iter 1) $M = M \setminus \{3\} = \{2, 4, 5, 6\}$

CC={1, 3}

En effet, $\Gamma^{-1}(3) = \{1\}$ et 1 est déjà exploré

$$D[3] = D[1] + \text{cout}(1, 3) = 0 + (-2) = -2$$

D=	0	∞	-2	∞	∞	∞
P=	1	-1	1	-1	-1	-1
	1	2	3	4	5	6

Iter 2) $M = M \setminus \{2\} = \{4, 5, 6\}$

CC={1, 2, 3}

En effet, $\Gamma^{-1}(2) = \{1, 3\}$ et 1 et 3 sont déjà été exploré

D=	0	-1	-2	∞	∞	∞
P=	1	3	1	-1	-1	-1
	1	2	3	4	5	6

$$D[2] = D[1] + \text{cout}(1, 2) = 0 + 1 = 1$$

$$D[2] = D[3] + \text{cout}(3, 2) = -2 + 1 = -1$$

Remarque : à cette étape, on aurait pu choisir de supprimer le sommet 5 au lieu de 2. La stratégie du choix est faite en fonction de l'ordre sur les sommets.

Iter3) $M = M \setminus \{5\} = \{4, 6\}$

CC={1, 2, 3, 5}

En effet, $\Gamma^{-1}(5) = \{3\}$ et 3 est déjà exploré

$$D[5] = D[3] + \text{cout}(3, 5) = -2 + 4 = +2$$

D=	0	-1	-2	∞	2	∞
P=	1	3	1	-1	3	-1
	1	2	3	4	5	6

Iter 4) $M = M \setminus \{6\} = \{4\}$

CC={1, 2, 3, 5, 6}

En effet, $\Gamma^{-1}(6) = \{2, 5\}$ et 2 et 5 sont déjà été exploré

$$D[6] = D[2] + \text{cout}(2, 6) = -1 + 3 = 2$$

$$D[6] = D[5] + \text{cout}(5, 6) = 2 + (-1) = 1$$

D=	0	-1	-2	∞	2	1
P=	1	3	1	-1	3	5
	1	2	3	4	5	6

Iter 5) $M = M \setminus \{4\} = \{\}$

CC = S

En effet, $\Gamma^{-1}(4) = \{2, 3, 6\}$ et 2, 3 et 6 sont déjà été exploré

$$D[4] = D[2] + \text{cout}(2, 4) = -1 + (-2) = -3$$

$$D[4] = D[3] + \text{cout}(3, 4) = -2 + 5 = 3$$

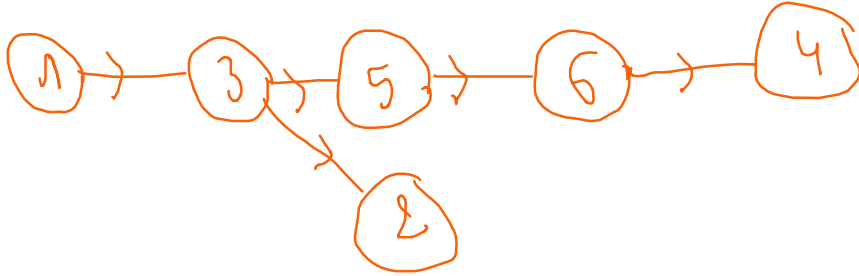
$$D[4] = D[6] + \text{cout}(6, 4) = 1 + (-5) = -4$$

D=	0	-1	-2	-4	2	1
P=	1	3	1	6	3	5
	1	2	3	4	5	6

Arborescence du PCC

PCC (1, 4)=-4

Chemin(1, 4)=1—3---5—6--4

**ALGORITHME DE FLOYD (COÛTS QUELCONQUES)***But*

Recherche d'un plus court chemin pour tout couple de sommets (s, t).

Principe

On suppose que les sommets du graphe sont numérotés de 1 à n.

Soit $S = \{1, 2, \dots, n\}$ l'ensemble des sommets du graphe.

On considère successivement les chemins de s vers t qui ne passent d'abord par aucun autre sommet,

Puis les chemins qui passent éventuellement par le sommet 1

Puis les chemins qui passent par 1 et 2

Etc

A l'étape k, on calcule le coût $D^k(s, t)$ d'un plus court chemin de s à t passant par des sommets inférieurs ou égaux à k.On note $\text{Pred}^k(s, t)$ le prédécesseur de t dans ce plus court chemin.

*Etapes de l'algorithme***Initialisation**

Pour tout sommet s et t de $G=(S, A, V)$

$$D[s, s] \leftarrow 0$$

$$\left. \begin{array}{l} D[s, t] \leftarrow \text{Coût}(s, t) \\ P[s, t] \leftarrow s \end{array} \right\} \text{ s'il existe un arc } (s, t) \in A$$

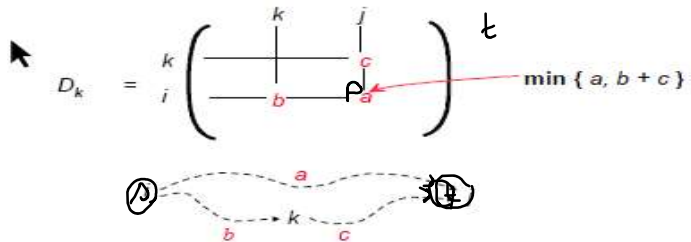
$$\left. \begin{array}{l} D[s, t] \leftarrow \infty \\ P[s, t] \leftarrow -1 \end{array} \right\} \text{ sinon}$$

Iterations

A l'étape k

$$D^k[s, t] \leftarrow \min(D^{k-1}[s, t], D^{k-1}[s, k] + D^{k-1}[k, t])$$

$$P[s, t] \leftarrow P[k, t]$$

**Test d'arrêt**

$$k > n$$

Algorithme de Floyd

//Calcul un plus court chemin entre tout couple de sommets (s, t) de G .

Procédure Floyd(G, D, P)

Début

// Initialisations

Pour s variant de 1 à n faire

Pour t variant de 1 à n faire

$$\left. \begin{array}{l} D[s, t] \leftarrow \text{Coût}(s, t) \\ P[s, t] \leftarrow s \end{array} \right\} \text{ s'il existe un arc } (s, t) \in A$$

$$\left. \begin{array}{l} D[s, t] \leftarrow \infty \\ P[s, t] \leftarrow -1 \end{array} \right\} \text{ sinon}$$

Finpour

$$D[s, s] \leftarrow 0$$

$$P[s, s] \leftarrow s$$

Finpour

// itérations

Pour k variant de 1 à n faire

Pour s variant de 1 à n faire

Pour t variant de 1 à n faire

 $\text{tmp} \leftarrow D[s, k] + D[k, t]$ Si $D[s, t] > \text{tmp}$ alors $D[s, t] \leftarrow \text{tmp}$ $P[s, t] \leftarrow P[k, t]$

Fsi

Fpour

Fpour

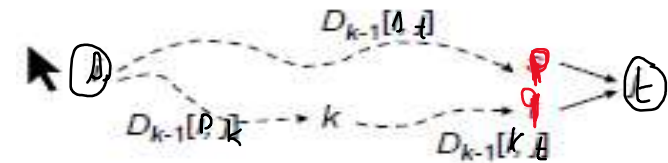
Fpour

Fin

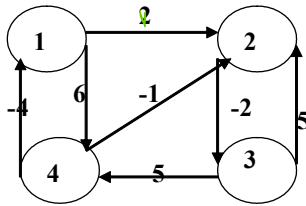
Matrice des Prédécesseurs

Mémorisation explicite des plus courts chemins du sommet s au sommet t

$P_k = P_k[s, t] = \text{Prédécesseur du sommet } t \text{ sur un plus court chemin de } s \text{ à } t \text{ dont les sommets intermédiaires sont tous } \leq k.$

Réurrence : $P_0[s, t] = s \quad \text{si } (s, t) \in A(G)$ $= -1 \quad \text{sinon}$ $P_k[s, t] = P_{k-1}[s, t] \text{ si } D^{k-1}[s, t] < (D^{k-1}[s, k] + D^{k-1}[k, t])$ $P_k[s, t] = P_{k-1}[k, t] \text{ si non}$ 

Exemple



Initialisation

$$K=0 \quad D = \begin{bmatrix} 0 & 2 & \infty & 6 \\ \infty & 0 & -2 & \infty \\ \infty & 5 & 0 & 5 \\ -4 & -1 & \infty & 0 \end{bmatrix} \quad P = \begin{bmatrix} 1 & 1 & -1 & 1 \\ -1 & 2 & -1 & -1 \\ -1 & 3 & 3 & 3 \\ 4 & 4 & -1 & 4 \end{bmatrix}$$

Itérations

$$K=1 \quad D = \begin{bmatrix} 0 & 2 & \infty & 6 \\ \infty & 0 & -2 & \infty \\ \infty & 5 & 0 & 5 \\ -4 & -2 & \infty & 0 \end{bmatrix} \quad P = \begin{bmatrix} 1 & 1 & -1 & 1 \\ -1 & 2 & -1 & -1 \\ -1 & 3 & 3 & 3 \\ 4 & 1 & -1 & 4 \end{bmatrix}$$

$$K=2 \quad D = \begin{bmatrix} 0 & 2 & 0 & 6 \\ \infty & 0 & -2 & \infty \\ \infty & 5 & 0 & 5 \\ -4 & -2 & -4 & 0 \end{bmatrix} \quad P = \begin{bmatrix} 1 & 1 & 2 & 1 \\ -1 & 2 & -1 & -1 \\ -1 & 3 & 3 & 3 \\ 4 & 1 & -2 & 4 \end{bmatrix}$$

$P[k, t]$

$$K=3 \quad D = \begin{bmatrix} 0 & 2 & 0 & 5 \\ \infty & 0 & -2 & 3 \\ \infty & 5 & 0 & 5 \\ -4 & -2 & -4 & 0 \end{bmatrix} \quad P = \begin{bmatrix} 1 & 1 & 2 & 3 \\ -1 & 2 & 3 & 3 \\ -1 & 3 & 3 & 3 \\ 4 & 1 & -2 & 4 \end{bmatrix}$$

$$K=4 \quad D = \begin{bmatrix} 0 & 2 & 0 & 5 \\ -1 & 0 & -2 & 3 \\ 1 & 3 & 0 & 5 \\ -4 & -2 & -4 & 0 \end{bmatrix} \quad P = \begin{bmatrix} 1 & 1 & 2 & 3 \\ 4 & 2 & 3 & 3 \\ 4 & 1 & 3 & 3 \\ 4 & 1 & 2 & 4 \end{bmatrix}$$

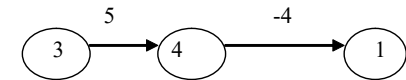
Exemples de chemins

$$K=4 \quad D = \begin{bmatrix} 0 & 2 & 0 & 5 \\ -1 & 0 & -2 & 3 \\ 1 & 3 & 0 & 5 \\ -4 & -2 & -4 & 0 \end{bmatrix} \quad P = \begin{bmatrix} 1 & 1 & 2 & 3 \\ 4 & 2 & 3 & 3 \\ 4 & 1 & 3 & 3 \\ 4 & 1 & 2 & 4 \end{bmatrix}$$

PCC entre 3 et 1 :

Distance de 3 à 1 est $D_4[3, 1]=1$

Chemin de 3 à 1 est :

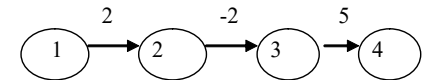


$$P_4[3, 1] = 4 \text{ et } P_4[3, 4] = 3$$

PCC entre 1 et 4 :

Distance de 1 à 4 est $D_4[1, 4]=5$

Chemin de 1 à 4 est :



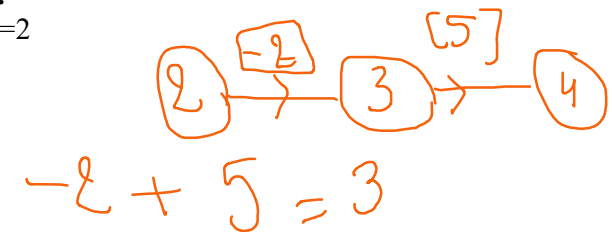
$$P_4[1, 4] = 3, P_4[1, 3] = 2 \text{ et } P_4[1, 2] = 1$$

PCC entre 2 et 4 :

Distance de 2 à 4 est $D_4[2, 4]=3$

Chemin de 2 à 4 est :

$$P_4[2, 4] = 3 \text{ et } P_4[2, 3] = 2$$



Détection de circuit absorbant

But

Détection de la présence de circuits absorbants en utilisant l'algorithme de Floyd.

Théorème

Soit D la matrice résultat obtenu en exécutant l'algorithme de Floyd.

G contient un circuit absorbant si et seulement si il existe un sommet s de G tel que $D[s, s] < 0$.

Preuve

❖ Si G a un circuit absorbant qui est une boucle de valuation $v_{ss} < 0$, à l'issue de l'exécution nous avons $D[s, s] < 0$.

Si G a un circuit absorbant qui n'est pas une boucle, il existe deux sommets s et t ($s \neq t$), et deux chemins $(s, \dots, t)$ de longueur L_{st} et $(t, \dots, s)$ de longueur L_{ts} tels que $L_{st} + L_{ts} < 0$.

A l'issue de l'exécution de l'algorithme de Floyd nous avons :

$$D[s, t] \leq L_{st}, D[t, s] \leq L_{ts} \text{ et } D[s, s] \leq D[s, t] + D[t, s].$$

Ainsi nous obtenons

$$D[s, s] \leq D[s, t] + D[t, s] \leq L_{st} + L_{ts} < 0.$$

❖ Soit s un sommet tel que $D[s, s] < 0$.

Soit D^k la valeur de la matrice D après l'itération k de la boucle .

Après l'initialisation

$D^0[s, s] = 0$ s'il n'y a pas de boucle d'évaluation strictement négative en s .

$D^0[s, s] < 0$ s'il existe une boucle d'évaluation strictement négative en s .

Pour tout couple de sommets s, t la suite des valeurs $D^0[s, t], D^1[s, t], \dots, D^n[s, t]$ est décroissante.

Il existe donc une itération k où $D^k[s, s] < 0$.

Si $k = 0$ il y a une boucle de valuation strictement négative en s qui constitue un circuit absorbant.

Si $k > 0$ nous avons $D^k[s, s] = D^{k-1}[s, t] + D^{k-1}[t, s] < 0$

Avec $D^{k-1}[s, t]$ la longueur d'un chemin du sommet s au sommet t

Et $D^{k-1}[t, s]$ la longueur d'un chemin du sommet t au sommet s .

Donc G contient un circuit un circuit de longueur strictement négative passant par s .